Índice

Objetivos de la sesión
Conocimientos y materiales necesarios
1. Preparando el entorno
  1.1. Creación de la máquina virtual
  1.2. Arrancando el sistema
2. Interfaz de comandos
  2.1. Directorio de trabajo
  2.2. Editando ficheros
  2.3. Permisos
  2.4. Operaciones básicas con ficheros
  2.5. Ayuda en línea
  2.6. Deteniendo el sistema
3. Control de versiones
4. Transferencia de ficheros

Introducción al entorno de trabajo

Área de Arquitectura y Tecnología de Computadores – Versión 1.1.11670, 13/09/2022

Objetivos de la sesión

En esta sesión se introduce al alumno el entorno de trabajo que se utilizará a lo largo de gran parte de las prácticas de laboratorio. Este entorno se basa en el sistema operativo GNU/Linux. Para esta sesión introductoria los principales objetivos son:

- Preparar el entorno e introducir el flujo de trabajo.
- Introducir el sistema operativo GNU/Linux y sus comandos básicos.
- Aprender a conectarse a máquinas GNU/Linux remotas y transferir ficheros.

Para conseguir estos objetivos se arrancará un sistema operativo Linux.

Conocimientos y materiales necesarios

Para poder realizar esta sesión, el alumno debe:

- Disponer de una máquina, física o virtual, con un sistema operativo Ubuntu Server 16.04.6 LTS 32 bits sobre el que se trabajará a lo largo del curso. En la parte inicial de la práctica se explica cómo crear la máquina virtual.

---

1. Preparando el entorno

Inicialmente debemos realizar unos preparativos para poder realizar las prácticas de la asignatura. Estos comprenden la instalación de la máquina GNU/Linux que vamos a utilizar y la definición del flujo de trabajo a seguir.

1.1. Creación de la máquina virtual

Para el desarrollo de las prácticas utilizaremos un sistema GNU/Linux. Puede servir tanto una máquina física como virtual, por lo que utilizaremos esta última opción por simplicidad.

Los profesores de la asignatura hemos preparado una máquina virtual con todas las herramientas necesarias que utilizaremos a lo largo de las prácticas. Concretamente, contiene un *driver* desarrollado específicamente que se carga automáticamente al arrancar el sistema operativo.

- Abre VirtualBox y elije la opción **Archivo › Importar servicio virtualizado** y selecciona el fichero que con la imagen de la máquina virtual que te indique el profesor. Opcionalmente, puedes descargar el fichero con la máquina virtual a tu ordenador.

  > Es posible que tengas que modificar antes la configuración de VirtualBox para crear la máquina en un directorio en el que tengas permiso de escritura. Asegúrate que las las máquinas se guardan en tu perfil de usuario usando la opción **Archivo › Preferencias**.

- Establece el nombre de la máquina virtual a `2AC` y deja el resto de opciones por defecto. Activa la opción ***Reinicializar la dirección MAC de todas las tarjetas de red***.
- Pulsa [**Importar**].

1.2. Arrancando el sistema

GNU/Linux es un sistema operativo multitarea y multiusuario, lo que indica que puede ejecutar varias tareas de distintos usuarios al mismo tiempo. Este operativo pertenece a la familia de operativos UNIX, por lo que presenta grandes similitudes con otros sistemas operativos como BSD o Mac OS X.

- Inicia la máquina con el sistema GNU/Linux instalado.

GNU/Linux ofrece dos modos de trabajo al usuario: modo gráfico y modo texto. En el modo gráfico se puede trabajar sobre el equipo a través de un escritorio similar al que puede encontrarse en otros sistemas operativos de ventanas. El modo texto ofrece la misma funcionalidad que el modo gráfico, con la desventaja de que todas las operaciones sobre el equipo deben realizarse a través de una interfaz de comandos. Este último modo es el utilizado habitualmente cuando nos conectamos de forma remota al equipo o bien cuando el sistema no tiene configurado el modo gráfico, típicamente en servidores, para ahorrar recursos en la máquina.

Dada la gran flexibilidad que ofrece la interfaz de comandos del operativo, y que no siempre estará disponible el modo gráfico cuando trabajemos con máquinas UNIX, este será nuestro entorno de trabajo. A continuación se introducirán los comandos básicos de la interfaz de comandos.

---

2. Interfaz de comandos

Cuando accedemos al sistema utilizando el modo texto de UNIX directamente aparece la interfaz de comandos del sistema operativo.

Puedes acceder a través de la consola de la máquina si estás ante ella (ya sea física o virtual), o bien a través de la red utilizando SSH. Necesitarás conocer la IP de la máquina para conectarte usando, por ejemplo, el programa PuTTY. En estas prácticas se asumirá este último caso, pues siempre podrá seguirse. En la consola de PuTTY puedes copiar texto seleccionando con el botón izquierdo del ratón y pegar pulsando el botón derecho.

- La máquina virtual se configura en modo NAT e incluye una regla que redirige el tráfico SSH, por lo que debes utilizar la dirección `localhost` y el puerto `60022` para conectarte a la máquina virtual con PuTTY.

  > La primera vez que te conectes se mostrará un aviso.

- Entra en sesión con el nombre de usuario `student` y la contraseña `student`.
- Cambia la contraseña siguiendo las instrucciones. Apunta la contraseña utilizada (idealmente, en tu gestor de contraseñas).

  > Ten cuidado cuando entres en sesión por primera vez para cambiar la contraseña. Inicialmente, se te pedirá la contraseña actual (`student`) para después introducir la nueva contraseña dos veces. Además, como medida de seguridad, la consola de Linux no muestra el eco local cuando introduces contraseñas, esto es, no verás los caracteres que escribes (no te aparecerán asteriscos). Después de cambiar la contraseña se cerrará la conexión. Podrás conectarte de nuevo, pero ya utilizando la nueva contraseña.

UNIX permite utilizar diferentes interfaces de comandos, aunque la más común es la interfaz `bash`. Esto tiene sus implicaciones a la hora de desarrollar tareas *batch*, es decir, cuando programamos para la propia interfaz de comandos a través de *scripts*.

> Un *script* es un fichero de texto que contiene comandos.

Dentro de la interfaz de comandos puedes utilizar la tecla `Tab` para autocompletar comandos (y nombres de ficheros) y las flechas de arriba y abajo para acceder al histórico de comandos introducidos en la consola.

2.1. Directorio de trabajo

Una vez abierta la interfaz de comandos se muestra el *prompt*, que es el texto que aparece al principio de la línea y que, por defecto, suele indicar el directorio de trabajo actual. Todos los usuarios tienen asignado un directorio de trabajo por defecto denominado `home`, que aparece representado en el *prompt* a través del carácter ~. El comando `pwd`[1] sirve para obtener el directorio actual de trabajo:

```
$> pwd
/home/student
```

El concepto de directorio de trabajo es de vital importancia, pues sobre él se realizarán por defecto todas las operaciones de acceso a ficheros, por ejemplo, crear un fichero de texto, eliminar un fichero, etc., cuando no se especifiquen rutas absolutas en los nombres de ficheros.

Al igual que otros sistemas de ficheros, en UNIX los ficheros se agrupan en directorios organizados de forma jerárquica[2]. A diferencia de los sistemas Windows, en UNIX no existen letras de unidad como `C:` o `D:`. En su lugar, todos los directorios cuelgan del directorio raíz, referenciado por `/`. Para cambiar el directorio de trabajo se utiliza el comando `cd`.

- Cambia el directorio de trabajo al directorio raíz.

  ```
  $> cd /
  ```

- Baja un nivel en la jerarquía de directorios al directorio `/etc`.

  ```
  $> cd etc
  ```

- Vete al directorio `/usr/include` especificando una ruta absoluta.

  ```
  $> cd /usr/include
  ```

- Retrocede al directorio anterior.

  ```
  $> cd -
  ```

- Sube un nivel en la jerarquía de directorios. Ten cuidado, hay un espacio entre el nombre del comando y los dos puntos.

  ```
  $> cd ..
  $> pwd
  /
  ```

- Vete al directorio `home`, es decir, al directorio de trabajo por defecto cuando arrancas la interfaz de comandos. Esto puede hacerse de varias formas.

  ```
  $> cd
  $> cd ~
  $> cd $HOME
  ```

Hay dos formas de especificar una ruta a través de la jerarquía de directorios: de forma absoluta o de forma relativa. Se puede especificar una ruta absoluta indicando el conjunto de directorios a través del que hay que bajar desde el raíz hasta llegar al fichero deseado. Por ejemplo, el siguiente comando utiliza la orden `cat`, que muestra el contenido de un fichero, especificando su ruta absoluta y, por lo tanto, funcionará independientemente del directorio donde esté situado el usuario.

```
$> cat /etc/hostname
2ac
```

También pueden utilizarse los directorios especiales `.` y `..` que referencian al directorio actual y al padre respectivamente para especificar una ruta relativa, es decir, a partir del directorio de trabajo actual. Observa que una ruta relativa nunca comienza por el directorio raíz `/` al contrario de lo que ocurría con una ruta absoluta.

```
$> cd
$> pwd
/home/student
$> cat ../../etc/hostname
2ac
$> cd /etc
$> cat hostname
2ac
$> cd ..
$> cat ./etc/hostname
2ac
$> cat etc/hostname
2ac
```

El contenido de un directorio puede listarse con el comando `ls`.

```
$> ls
bin   cdrom  etc   initrd.img  media  opt   rofs  sbin     srv  tmp  var
boot  dev    home  lib         mnt    proc  root  selinux  sys  usr  vmlinuz
```

La gran mayoría de sistemas UNIX organizan sus sistemas de ficheros de forma parecida, de tal forma que presentan directorios colgando del raíz con misiones específicas:

- `/home`. Es el directorio donde se ubican los directorios de los usuarios. Típicamente existe un directorio por usuario.
- `/bin`. Contiene comandos que pueden invocarse, tales como el comando `pwd`.
- `/dev`. En él se ubican los ficheros especiales que permiten acceder a los dispositivos del computador y periféricos.
- `/etc`. Contiene la mayor parte de los ficheros para la configuración del sistema y de los programas instalados.
- `/proc`. Contiene ficheros con información del sistema.

2.2. Editando ficheros

Para editar ficheros de texto a través de la consola podemos utilizar editores como `nano`.

Vamos a editar un fichero de texto utilizando el editor en modo consola. Ten cuidado con el directorio de trabajo, pues es donde se crea el fichero resultante.

- Abre el editor indicándole que vamos a editar un nuevo fichero de nombre `1-1texto.txt`. La opción `-c` hace que el editor muestre los números de línea en la parte inferior.

  ```
  $> cd
  $> nano -c 1-1texto.txt
  ```

- Escribe algo en el fichero.
- Guarda el fichero. Para ello utiliza la combinación `Ctrl`+`o` y pulsa `ENTER` para confirmar el nombre del fichero.
- Sal del editor utilizando la combinación `Ctrl`+`x`. Como se ve en la figura, este editor muestra también en la parte inferior una ayuda con el significado de diferentes combinaciones de teclas, donde `^` indica la tecla `Ctrl`.

Como cualquier otro comando, se le puede indicar al editor un nombre de fichero incluyendo una ruta relativa o absoluta. En el ejemplo anterior se especificó una ruta relativa, es decir, se le indicó al editor que creara el fichero en el directorio de trabajo actual, pero también puede especificarse una ruta absoluta con el fichero.

- Abre el fichero recién creado especificando su ruta absoluta.

  ```
  $> nano -c /home/student/1-1texto.txt
  ```

- Sal del editor.

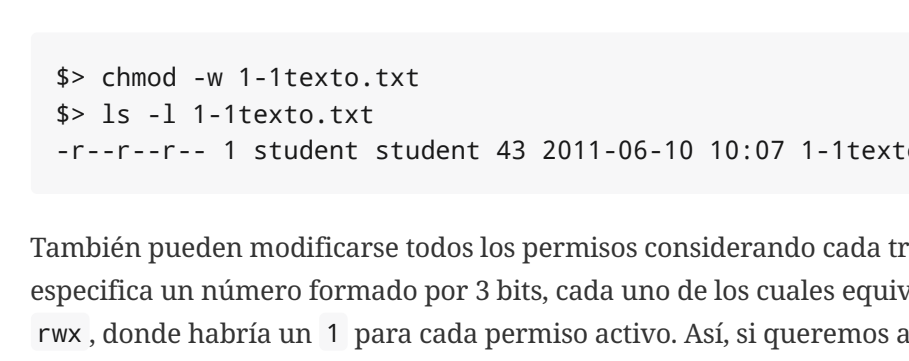
2.3. Permisos

Como ocurre en otros sistemas operativos multiusuario, UNIX etiqueta cada entrada en el sistema de ficheros (fichero, directorio o enlace simbólico) con una serie de permisos. En concreto, existen tres tipos de permisos, representados por distintos caracteres cuando se hace un listado de un directorio: permiso de lectura (`r`), de escritura (`w`) y de ejecución (`x`). Además, estos permisos se asignan en tres niveles distintos: al usuario propietario del fichero (directorio o enlace simbólico), al grupo propietario y al resto de usuarios del sistema.

Veámoslo con un ejemplo que utiliza la opción `-l` del comando `ls` para mostrar, además del nombre de los ficheros, sus atributos (permisos, propietario, etc.):

```
$> ls -l
total 12
drwxr-xr-x 2 student student 40 2011-06-10 10:42 directorio
-rw-rw-r-- 1 student student 43 2011-06-10 10:07 1-1texto.txt
-rw-r--r-- 1 student root   16 2011-06-10 10:40 otro.txt
-rwxr-xr-x 1 student student 18 2011-06-10 10:39 script
```

La información que se muestra en el listado está organizada en columnas con el siguiente significado:



- rwx r-x r-x 1 student student 18 2011-06-10 10:39 script
Grupo propietario
Usuario propietario
Permisos del resto de usuarios
Permisos del grupo propietario
Permisos del usuario propietario
Tipo de fichero

Figura 1. Significado de los permisos en UNIX

La primera entrada del listado se corresponde con un directorio, de ahí el carácter `d`, el resto de entradas son ficheros regulares, de ahí que muestren el carácter `-` en el tipo de fichero. Los enlaces simbólicos muestran el carácter `l`.

El usuario `student` es el propietario de las cuatro entradas que aparecen en el listado anterior. Además, este usuario tendrá permisos de lectura y escritura sobre todos los ficheros, mientras que tendrá además permisos de ejecución sobre el directorio y el fichero `script`.

Los permisos de un fichero pueden cambiarse utilizando el comando `chmod`. Por ejemplo, se puede eliminar el permiso de escritura sobre el fichero `1-1texto.txt` de la siguiente forma:

```
$> chmod -w 1-1texto.txt
$> ls -l 1-1texto.txt
-r--r--r-- 1 student student 43 2011-06-10 10:07 1-1texto.txt
```

También pueden modificarse todos los permisos considerando cada tripleta como un número binario. De esta forma, se especifica un número formado por 3 bits, cada uno de los cuales equivale a la representación en binario de los permisos `rwx`, donde habría un `1` para cada permiso activo. Así, si queremos asignar los siguientes permisos:

- Usuario propietario: todos los permisos ⇒ `rwx` ⇒ `111b` ⇒ `7d`.
- Grupo propietario: lectura y escritura ⇒ `rw-` ⇒ `110b` ⇒ `6d`.
- Resto de usuarios: solo lectura ⇒ `r--` ⇒ `100b` ⇒ `4d`.

```
$> chmod 764 1-1texto.txt
$> ls -l 1-1texto.txt
-rwxrw-r-- 1 student student 43 2011-06-10 10:07 1-1texto.txt
```

En ocasiones, igual que ocurre en otros sistemas operativos, es necesario tener los privilegios de administrador para realizar una tarea o ejecutar un comando. El usuario administrador en sistemas UNIX es habitualmente el usuario `root` (o superusuario). Sin embargo, cuando entramos en sesión rara vez lo hacemos como usuario administrador.

- Comprueba con qué usuario estás dentro del sistema. Utiliza el comando `whoami`.

  ```
  $> whoami
  student
  ```

Los sistemas UNIX ofrecen la posibilidad de ejecutar cualquier comando o programa con los privilegios del usuario *root* utilizando el comando `sudo`. Para ello el usuario o alguno de los grupos a los que pertenece debe aparecer en el fichero `/etc/sudoers`. Por ejemplo, para actualizar el sistema o instalar nuevos paquetes puede utilizarse el comando `apt-get`. Aunque este comando puede ser iniciado por cualquier usuario, durante su ejecución se requieren permisos de *root*. Vamos a instalar en nuestra máquina las últimas actualizaciones disponibles.

> Se utiliza el término paquete en UNIX para denominar el software que puede instalarse (programas, bibliotecas, etc.).

- Prueba a actualizar la lista de paquetes instalables invocando el comando `apt-get`. Verás que ocurre un error, pues no tienes los permisos adecuados.

  ```
  $> apt-get update
  ```

- Invoca ahora el comando con permisos de superusuario.

  ```
  $> sudo apt-get update
  ```

- Actualiza el sistema. El siguiente comando actualiza todos los paquetes para los que exista una versión más moderna.

  ```
  $> sudo apt-get upgrade
  ```

Recuerda el uso del comando `sudo`, ya que se utilizará a menudo a lo largo del curso.

2.4. Operaciones básicas con ficheros

Hasta ahora hemos visto cómo crear y editar ficheros de texto, listar el contenido de los directorios del sistema o cambiar el directorio actual de trabajo. Ahora veremos operaciones básicas como copiar, mover, renombrar o eliminar ficheros.

- Crea un directorio de nombre `dir` que cuelgue de tu directorio `home`. Para ello debes utilizar el comando `mkdir`, que como el resto de comandos puede recibir una ruta relativa o absoluta.

  ```
  $> mkdir ~/dir
  ```

- Haz una copia del fichero `1-1texto.txt` dentro del directorio recién creado que tenga el nombre `1-1copia1.txt`. Para ello debes utilizar el comando `cp`. Observa el ejemplo de invocación del comando e intenta entender qué significan los parámetros que se le pasan. Si tienes alguna duda pregúntale al profesor.

  ```
  $> cd ~/dir
  $> cp ../1-1texto.txt ./1-1copia1.txt
  ```

- Renombra el fichero `1-1copia1.txt` a `1-1copia2.txt` al tiempo que lo mueves a tu directorio `home` utilizando el comando `mv`. Este comando sirve tanto para mover como para renombrar ficheros.

  ```
  $> mv 1-1copia1.txt ../1-1copia2.txt
  ```

- Elimina el fichero `1-1copia2.txt` utilizando el comando `rm`. Mucho cuidado con este comando, pues no solicita confirmación cuando se borran los ficheros.

  ```
  $> cd ..
  $> rm 1-1copia2.txt
  ```

- Elimina el directorio `dir` que creamos al principio con el comando `rmdir`. Este comando requiere que el directorio esté vacío.

  ```
  $> rmdir dir
  ```

2.5. Ayuda en línea

GNU/Linux es un sistema operativo eminentemente orientado a usuarios desarrolladores de aplicaciones en C. De hecho, las distribuciones suelen incluir el compilador y el entorno de desarrollo, entre otras ayudas a la programación. Una de ellas es la ayuda en línea, que cubre aspectos de programación en C, así como comandos básicos de la interfaz de comandos.

La ayuda en línea de GNU/Linux y, en general, de los sistemas operativos UNIX se ofrece a través de manuales organizados en categorías. Cada una de estas categorías se numera desde el 1 hasta el 8 y comprenden comandos generales de línea de comandos (categoría 1) o funciones de biblioteca invocables desde un programa escrito en el lenguaje C (categoría 3).

Puedes acceder a la ayuda de cualquier comando utilizando la orden `man` seguida del nombre del comando.

- Consulta el manual acerca del comando `ls`. Puedes salir del manual pulsando `Q`.

  ```
  $> man ls
  ```

- También puedes especificar directamente la categoría donde consultar el manual. Consulta ahora el manual de la función `printf` de C. Esta función sirve para mostrar información en pantalla tal como se verá más adelante.

  ```
  $> man 3 printf
  ```

  En el manual se muestra información útil sobre cómo invocar la función y qué ficheros de cabecera hay que incluir para poder invocarla.

- Sal del manual.

La indicación explícita de la categoría a consultar resulta útil en el caso anterior, pues también existe un comando de la línea de comandos de nombre `printf`. Si no se hubiese especificado la categoría, se mostraría la ayuda del comando de la línea de comandos en lugar de la función de C. Puedes probar a invocar el comando anterior sin indicar la categoría y verás el resultado.

> Existe también un comando de la línea de comandos llamado `printf`.

La ayuda se puede consultar también a través de buscadores de internet, que habitualmente indexan los manuales. Generalmente, el resultado de consultar la ayuda desde la línea de comandos o desde un buscador lleva a un resultado equivalente.

2.6. Deteniendo el sistema

Como con cualquier otro sistema operativo, al acabar nuestro trabajo debemos detener el sistema.

- Apaga la máquina virtual usando el siguiente comando.

  ```
  $> sudo poweroff
  ```

  > Siempre debes utilizar esta comando para apagar la máquina virtual. Nunca lo hagas a través de la opción *Apagar máquina virtual* de VirtualBox, pues equivaldría a desconectar la máquina de la alimentación eléctrica. Sí podrías hacerlo con la opción *Enviar señal de apagado*.

- Vuelve a arrancar la máquina virtual.

  > Puede utilizarse el comando `reboot` para reiniciar la máquina, esto es, apagarla y volver a arrancarla.

---

3. Control de versiones

Cuando se trabaja con código fuente (y en general con cualquier tipo de documento que sufre una cierta evolución) se antoja muy recomendable el uso de los sistemas de control de versiones. En los casos de proyectos software de envergadura, resulta entonces imprescindible.

Estos sistemas permiten llevar registro de todos los cambios que se producen en el código (quién, qué y cuándo) de forma automática. A lo largo de esta asignatura utilizaremos *git* como sistema de control de versiones, si bien un flujo de trabajo mínimo. Concretamente, utilizaremos un repositorio[3] *git* en la nube como sistema de *backup* de los ficheros que vayamos generando.

- Configura *git* con tu nombre de usuario y tu correo electrónico. Estos serán los datos que aparecerán en el historial de cambios.

  ```
  $> git config --global user.name "<usuario>"
  $> git config --global user.email "<correo>"
  ```

  > No pongas los símbolos "<" y ">": se utilizan para indicar que no debes escribir esa palabra literalmente, pero sí las comillas.

- Regístrate en Bitbucket, que es un servicio de repositorios *git* gratuito. Puedes utilizar tu *UO* como nombre de usuario.
- Crea un nuevo repositorio pinchando en el símbolo `+` dentro del menú y eligiendo la opción **Repositorios › Crear repositorio**. Dale como nombre `2ac-plX-uo<xxxxxx>`, dejando el resto de opciones por defecto. Sustituye `plX` por el identificador de tu grupo de laboratorio.
- Dale a tu profesor de prácticas acceso al repositorio de escritura en la sección de **Settings › User and group access**. El profesor te indicará su nombre de usuario.

  Para poder acceder al repositorio remoto en Bitbucket desde la interfaz de comandos es necesario crear lo que en la terminología de Bitbucket se conoce como una *app password*.

- Pulsa sobre el avatar que se encuentra en la esquina superior derecha de la página de Bitbucket. Inicialmente, el avatar consiste en la inicial de tu nombre y de tu primer apellido. Selecciona `Personal settings` y en el menú de la izquierda `App passwords`. A continuación pulsa el botón [**Create app password**]. Debes indicar una etiqueta de tu elección que te permitirá distinguirla de otras *app passwords* que puedas crear en el futuro. Además, debes activar, al menos, todos los permisos de la sección `Repositories`.
- Pulsa el botón [**Create**] para crear la *app password* con los datos introducidos.
- Copia y pega la *app password* en un documento de texto y guárdala en lugar seguro, pues necesitarás usarla en breve.
- Navega de vuelta en la página de Bitbucket a tu repositorio y pincha en el botón [**Clone**]. Copia el texto que aparece en la ventana emergente (es un comando).
- Desde la interfaz de comandos de Linux colócate en tu directorio `home` y ejecuta el comando que acabas de copiar para clonar el repositorio de Bitbucket en tu máquina. Debería ser parecido al siguiente, donde `<usuario>` representa tu nombre de usuario en Bitbucket.

  ```
  $> git clone https://<usuario>@bitbucket.org/<usuario>/2ac-plX-uo<xxxxxx>.git
  ```

- Observarás que te pide una contraseña. Debes introducir la *app password* que has copiado al archivo de texto. Recuerda que puedes pegarla en la ventana de `putty` pulsando el botón derecho del ratón. Aunque aparentemente no se muestre pegada sí que lo estará, simplemente se oculta por cuestiones de seguridad. Pulsa la tecla `Intro` y observa cómo se clona el repositorio localmente.
- Cambia al directorio `2ac-plX-uo<xxxxxx>` donde se ha copiado el repositorio.

  > De aquí en adelante, crearás siempre los ficheros de las prácticas dentro de este directorio.

- Para evitar introducir la *app password* cada vez que accedas al repositorio remoto en Bitbucket, se puede hacer que *git* la almacene en el repositorio local. Simplemente ejecuta dentro del directorio del repositorio la siguiente orden:

  ```
  $> git config credential.helper store
  ```

  La próxima vez que intentes acceder al repositorio remoto en Bitbucket te pedirá la *app password* una única vez.

El flujo de trabajo que seguiremos siempre será:

1. Al comienzo de cada sesión de prácticas traer del repositorio de Bitbucket los cambios que se hubiesen realizado con anterioridad.

   > No ejecutes estas órdenes todavía. Esto es sólo una explicación.

   ```
   $> git pull
   ```

2. Crear o modificar los ficheros que desarrollemos.
3. Añadir los ficheros o los cambios que hayamos realizado que queremos que estén bajo el control de versiones (de los que queremos *backup*) con la siguiente orden, donde `<fichero>` es el nombre del fichero añadido o modificado.

   ```
   $> git add <fichero>
   ```

4. Confirmar los cambios realizados, de forma que se registren en el sistema de control de versiones. Se solicitará un mensaje en el que indicarás una breve descripción de los cambios realizados. Normalmente realizaremos varios *commits* en cada sesión de prácticas para crear sucesivas versiones en el repositorio.

   ```
   $> git commit
   ```

   > Se abrirá un editor para que puedas introducir la descripción de los cambios realizados. Guarda el fichero para confirmar el *commit*.

5. Cuando acabemos de trabajar, enviar los cambios hacia el repositorio de Bitbucket.

   ```
   $> git push
   ```

Resultaría conveniente crear un subdirectorio para cada sesión de prácticas dentro del directorio con el repositorio *git*.

A modo de ejemplo, vamos a añadir un fichero con la descripción del contenido del repositorio que aparecerá en la interfaz gráfica de Bitbucket.

- Crea un fichero de texto de nombre `README.md` y añade el siguiente texto.

  ```
  # General #

  Repositorio para la asignatura Arquitectura de Computadores.
  ```

- Una vez guardado el fichero, puedes comprobar cómo git detecta que existe un nuevo fichero que no está bajo el control de versiones.

  ```
  $> git status
  ```

- Indícale a git los cambios que se han producido (se dice añadir los cambios al índice). Esto le indica a git qué cambios debe incluir en la siguiente versión.

  ```
  $> git add README.md
  ```

- Comprueba de nuevo el estado del repositorio. Ahora debería aparecer que el nuevo fichero está listo para incluirse en la siguiente versión.

  ```
  $> git status
  ```

- Confirma los cambios indicando una descripción de los mismos.

  ```
  $> git commit
  ```

  > Debes proporcionar una descripción de los cambios de forma obligatoria. Recuerda guardar el fichero al salir para reflejar esta descripción.

- Envía los cambios al respositorio remoto (en Bitbucket).

  ```
  $> git push --set-upstream origin master
  ```

  > Una vez que ejecutes este comando por primera vez, ya no es necesario indicar la opción `--set-upstream origin master` en posteriores llamadas; basta con llamar al comando `git push`.

- Tras esto, puedes comprobar cómo la interfaz de Bitbucket te muestra el cambio que se ha producido y te permite inspeccionar el contenido del respositorio.
- También puedes comprobar el histórico de cambios en tu repositorio local utilizando el siguiente comando.

  ```
  $> git log
  ```

Puedes encontrar más información del funcionamiento de git en este libro online.

---

4. Transferencia de ficheros

Un problema que se plantea cuando se utilizan máquinas virtuales es cómo transferir ficheros entre la máquina cliente en la que iniciamos sesión local (anfitrión) y la máquina virtual (invitado).

Vamos a utilizar un programa que hace uso de una conexión SSH para transferir ficheros entre las dos máquinas. Se trata de WinSCP, que es un programa portable que no requiere instalación.

- Si no está ya instalado en tu equipo anfitrión, descarga la versión portable del WinSCP desde su página de descarga.
- Ejecuta el programa y establece una conexión SFTP con la máquina remota (misma dirección y puerto que con PuTTY). Fíjate que este procedimiento es el mismo independientemente de que la máquina remota sea virtual o física.
- Verás que la interfaz de WinSCP te muestra dos árboles de directorios: el local (a la izquierda) y el remoto (a la derecha). Prueba a copiar algún fichero hacia la máquina remota y a la local sin más que arrastrar los ficheros entre ambas ventanas.

---

[1] Ten cuidado al introducir comandos porque la interfaz de comandos distingue entre mayúsculas y minúsculas.
[2] En Windows es común denominar carpetas a los directorios.
[3] Un repositorio es un contenedor con el historial de cambios de los ficheros a él asociados.